UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería
Ingeniería de Software

Sistemas Distribuidos
TA-IND-01 — Informe académico individual

Análisis crítico de TiDB frente a YugabyteDB

Arquitectura, consistencia y tolerancia a fallos

Estudiante: Manotoa Alava Carlos Alexander

Curso: Séptimo semestre, paralelo A

Fecha: 13 de noviembre de 2026

Repositorio:
`https://github.com/BelindaTZ/TA-IND-01-Informe-Acad-mico-Individual---CGMT`

Resumen

Este informe presenta un análisis de TiDB y una comparación con YugabyteDB, dos bases de datos que buscan mantener las transacciones SQL mientras distribuyen la información entre varios nodos. Para comprender su funcionamiento se revisan sus componentes, las transparencias de distribución, CAP, PACELC, los modelos de comunicación y la tolerancia a fallos. También se realizó una práctica local con TiDB 7.5.0 y Docker Compose, formada por un servidor TiDB, un PD y tres TiKV. En las capturas se observa el inicio del entorno, la creación de una tabla, la terminación del contenedor tidb-tikv1 y una inserción aceptada después de esa acción. Este resultado permite relacionar la teoría con una operación real, aunque no demuestra que el servicio nunca se haya interrumpido ni permite medir su tiempo de recuperación. La comparación muestra que ambos sistemas usan Raft para mantener réplicas consistentes, mientras organizan de forma distinta el almacenamiento y las marcas de tiempo. Se concluye que TiDB puede mantener operaciones ante la caída probada, pero su disponibilidad también depende de componentes que no se pusieron a prueba y de cómo se configure el entorno.

1. Introducción

Las bases de datos relacionales permiten trabajar con tablas y mantener garantías transaccionales. Cuando una aplicación crece, también necesita repartir almacenamiento y procesamiento entre varias máquinas. Aquí aparecen los sistemas SQL distribuidos, que buscan conservar esa forma de trabajar mientras coordinan datos y operaciones entre diferentes nodos. TiDB y YugabyteDB son dos herramientas dentro de esta categoría, pero la manera en que organizan sus componentes influye en su comportamiento cuando ocurre un fallo [1], [2].

TiDB separa el procesamiento SQL del almacenamiento y agrega una capa llamada TiFlash. Esta capa organiza los datos por columnas para facilitar consultas analíticas; así se pueden combinar las operaciones diarias con el análisis de información, lo que se conoce como arquitectura HTAP. La revisión de Song y sus colaboradores explica que combinar ambos trabajos también introduce problemas de sincronización, frescura de datos y uso de recursos [3].

El sistema tiene aplicaciones fuera del entorno de pruebas. Por ejemplo, PingCAP presenta el caso de WeBank, donde reporta más de 80 clústeres y más de 1,3 PB de datos. Estas cifras corresponden al caso publicado por el proveedor y sirven para ubicar su escala de uso; no son resultados obtenidos en esta práctica [4].

TiDB permite observar de forma local cómo se relacionan los componentes de una base distribuida, por lo que resulta adecuado para este trabajo. YugabyteDB se utiliza como comparación por su interfaz SQL vinculada con PostgreSQL y su almacenamiento distribuido DocDB. El propósito del informe es comprender esas decisiones y juzgar sus límites, apoyándome en documentación oficial, artículos recientes y las capturas del ejercicio que realicé.

2. Arquitectura del sistema

Para entender cómo responde el sistema ante una caída, primero hay que conocer qué hace cada componente. En esta sección se presenta la organización de TiDB y después la de YugabyteDB. Con esa base se puede seguir el recorrido de una consulta y comprender por qué algunos servicios necesitan coordinarse antes de confirmar una operación.

2.1. TiDB y la separación de sus componentes

TiDB divide el trabajo en partes con funciones diferentes. TiDB Server recibe las consultas SQL, las analiza y prepara el plan de ejecución. Aquí se procesa la consulta; las tablas se guardan de forma persistente en la capa de almacenamiento. Esto permite ampliar la capacidad de cómputo sin que cada servidor SQL tenga que conservar una copia local completa de las tablas [1].

Placement Driver (PD) se encarga de conocer cómo están distribuidos los datos y de planificar movimientos de Regions. También proporciona marcas de tiempo mediante el TimeStamp Oracle, conocido como TSO. Se puede entender como una parte de coordinación del sistema, aunque no procesa por sí solo todas las consultas ni almacena los datos de las tablas [5].

TiKV guarda la información como pares clave–valor y la divide en rangos llamados Regions. Cada Region tiene sus propias réplicas y forma un grupo Raft. Un servidor TiKV puede alojar muchas Regions, por lo que no se debe confundir un nodo completo con una única réplica o con un único grupo de consenso [6].

TiFlash es el componente opcional que mantiene una representación por columnas. Recibe cambios de TiKV como learner de Raft, sin participar en la mayoría que confirma las escrituras. Su actualización es asíncrona, pero antes de responder una lectura comprueba que tenga el avance necesario para el timestamp solicitado. Por eso, una consulta puede tener que esperar a que la réplica se ponga al día [7].

El TSO ayuda a ordenar las transacciones. Este servicio utiliza una parte física y otra lógica en sus marcas de tiempo. Su funcionamiento agrega una dependencia de coordinación, pero no significa que cada sentencia SQL produzca siempre una petición independiente al PD. El costo depende de la operación y de las optimizaciones que utilice el sistema [5].

2.2. YugabyteDB y sus relojes híbridos

YugabyteDB organiza sus datos mediante DocDB, un almacenamiento distribuido construido sobre RocksDB. Divide las tablas en fragmentos llamados tablets y los replica mediante grupos Raft. Los servicios YB-TServer alojan las tablets y YB-Master mantiene funciones de catálogo y coordinación. También existe una separación lógica entre la capa de consultas y la de almacenamiento, aunque puedan ejecutarse en el mismo nodo [2].

Su interfaz YSQL reutiliza la capa de consultas de PostgreSQL y la conecta con DocDB. Esto facilita trabajar con una interfaz conocida, pero requiere comprobar la compatibilidad de cada aplicación. La interfaz SQL y el motor que mantiene los datos cumplen funciones distintas dentro del conjunto.

Para asignar marcas de tiempo, YugabyteDB usa relojes lógicos híbridos, llamados HLC. Estos combinan la hora física con una parte lógica y se actualizan cuando los nodos intercambian información. Así se mantiene un orden útil para las transacciones sin depender del mismo esquema de TSO centralizado de TiDB. Aun así, continúan existiendo límites de sincronización, tiempos seguros de lectura y posibles reintentos [8].

La diferencia principal está en cómo se organiza esa coordinación. Tener HLC no demuestra por sí solo que una consulta sea más rápida, ni elimina la necesidad de esperar a una mayoría de réplicas para determinadas operaciones.

2.3. Del boceto inicial al diagrama de arquitectura

La Figura 1 parte del boceto del Anexo A y del diagrama en draw.io. Se distinguen los TiKV votantes y TiFlash como learner.

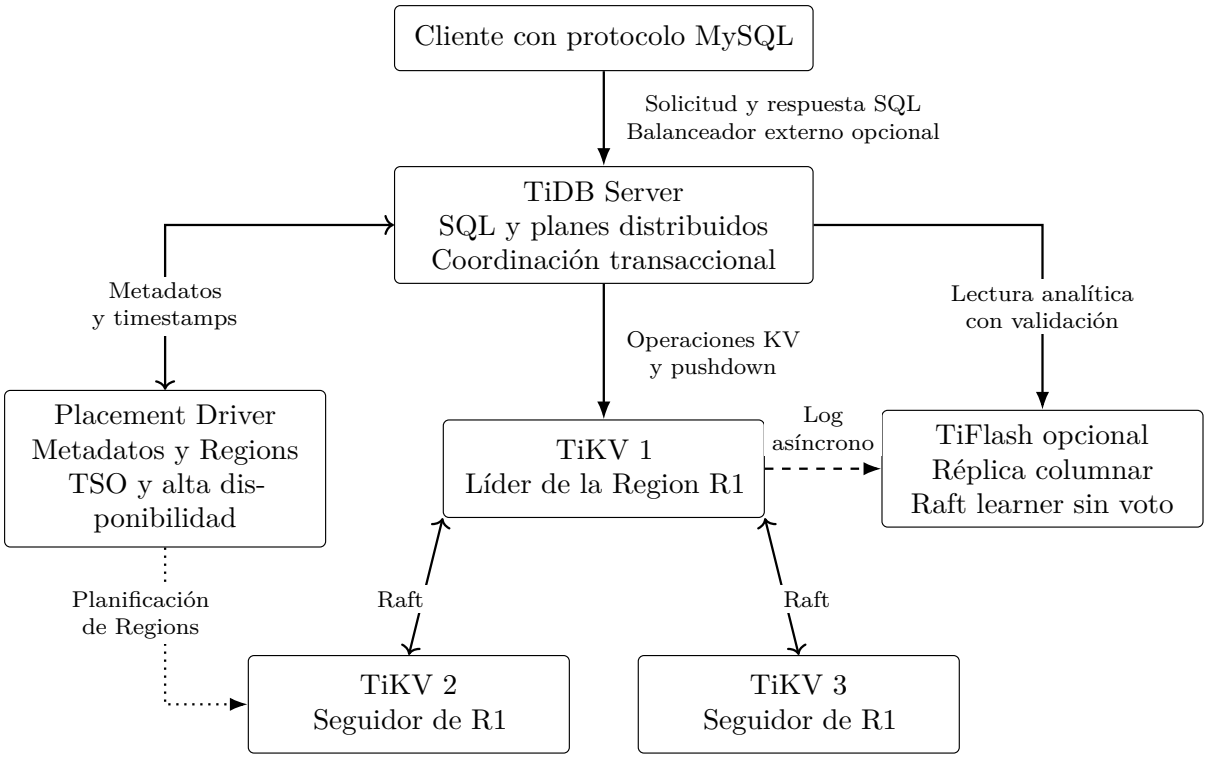


Figura 1: Arquitectura conceptual de TiDB basada en el boceto y el diagrama del trabajo. R1 representa una Region con tres réplicas votantes.

En el dibujo se puede seguir la consulta desde el cliente hasta TiDB Server y luego hacia el almacenamiento. Las flechas de Raft corresponden a la replicación de una Region; otros grupos pueden tener líderes diferentes. TiFlash y el balanceador forman parte de la explicación general, pero no estuvieron incluidos en el despliegue Docker de la práctica.

3. Análisis con los conceptos de la Unidad 1

Una vez revisada la arquitectura, los conceptos de la unidad permiten explicar qué parte de la distribución queda oculta y qué ocurre cuando la comunicación o un componente falla. Aquí se relacionan esos conceptos con las decisiones de TiDB y YugabyteDB, procurando distinguir las garantías del diseño de lo que se pudo comprobar en el ejercicio.

3.1. Transparencias de distribución

Las transparencias permiten que el usuario trabaje con el sistema sin controlar todos sus detalles internos. En los sistemas analizados aparecen en diferentes niveles. Algunas ocultan la ubicación de los datos, mientras que otras solo funcionan bajo ciertas condiciones, como conservar suficientes réplicas disponibles.

Transparencia de acceso. En TiDB se utiliza un cliente compatible con MySQL; en YugabyteDB, YSQL ofrece una interfaz vinculada con PostgreSQL. El usuario puede escribir consultas sin conocer el protocolo interno de replicación. Esto facilita el acceso, aunque la compatibilidad debe revisarse cuando se utilizan funciones específicas de cada motor [1], [2].

Transparencia de ubicación. Al consultar una tabla no se indica en qué TiKV o tablet se encuentra cada fila. El sistema se encarga de localizar los datos. Como usuario se conoce el punto de conexión, pero no hace falta elegir manualmente el servidor que contiene cada registro.

Transparencia de migración. Los datos pueden redistribuirse entre nodos sin cambiar el nombre de la tabla ni reescribir las consultas. En TiDB, PD participa en la planificación de Regions. Este movimiento tiene un costo de trabajo y comunicación, aunque la aplicación no tenga que hacerlo de forma manual [6].

Transparencia de reubicación. Cuando los datos o el liderazgo cambian durante el uso del sistema, la aplicación intenta seguir trabajando con la misma interfaz. Esto no garantiza que una conexión TCP pueda trasladarse sin efectos a otro servidor. Algunas operaciones pueden necesitar un reintento o una reconexión.

Transparencia de replicación. La aplicación realiza una inserción y el sistema coordina sus réplicas. El usuario no ejecuta una inserción distinta por cada copia. Aquí también es importante diferenciar replicación de una copia de seguridad histórica: una eliminación válida puede propagarse a las réplicas y seguir siendo necesario un respaldo independiente [9], [10].

Transparencia de concurrencia. Varios usuarios pueden realizar operaciones al mismo tiempo, mientras los mecanismos transaccionales controlan las versiones y los conflictos. MVCC permite trabajar con versiones de los datos; las marcas de tiempo ayudan a establecer qué información corresponde a cada lectura. Los relojes, por sí solos, no resuelven todo el control de concurrencia [8], [10].

Transparencia de fallos. Una caída puede quedar parcialmente oculta si los componentes restantes conservan las condiciones necesarias para continuar. También puede llegar al usuario como una espera o un error. Esta transparencia tiene límites, especialmente cuando se pierde la mayoría de un grupo o falla un servicio del que depende la operación [11], [12].

3.2. CAP y PACELC

CAP y PACELC ayudan a analizar las decisiones que se toman cuando los nodos deben coordinarse. El primero se centra en el problema que aparece durante una partición de red; el segundo permite considerar también el costo de mantener ciertas garantías cuando la red está funcionando.

3.2.1. Teorema CAP

Una partición ocurre cuando partes del sistema dejan de comunicarse. En esa situación, una operación no puede contar a la vez con disponibilidad en todos los lados y con un único orden consistente de lecturas y escrituras. La consistencia de CAP se relaciona con una visión linealizable de las operaciones y no debe confundirse con cualquier nivel de aislamiento SQL ni con la integridad de ACID [10].

En el camino de escritura de TiKV, el líder necesita que una mayoría de las réplicas del grupo participe. Si tres votantes quedan separados en un grupo de dos y otro de uno, la parte con dos puede continuar si conserva las demás dependencias de la transacción. El votante aislado no puede confirmar escrituras por su cuenta. En este alcance se entiende la clasificación CP [9], [11].

YugabyteDB también usa replicación síncrona mediante Raft en sus grupos principales. Por tanto, el requisito de mayoría también limita sus escrituras consistentes ante una partición. Esto permite compararlos sin suponer que uno mantiene disponibilidad completa en cualquier lado de la red [12].

3.2.2. Modelo PACELC

PACELC plantea que, si existe una partición, hay un compromiso entre disponibilidad y consistencia; en condiciones normales también se debe considerar la relación entre latencia y consistencia. Esperar confirmaciones de otros nodos puede aumentar el tiempo de respuesta, incluso cuando no existe una falla.

En este informe se interpreta que las escrituras consistentes de ambos sistemas conservan ese costo de coordinación. El uso de HLC en YugabyteDB no basta para decir que siempre prioriza latencia sobre consistencia. Sus read replicas asíncronas son una opción diferente que permite acercar lecturas, con condiciones de frescura que deben analizarse aparte [8], [13].

La práctica realizada termina un contenedor de almacenamiento. Con eso se observa una caída concreta, pero no se provoca una partición de red ni se mide el compromiso de latencia de PACELC. Para probar esas situaciones harían falta otros controles y mediciones.

3.3. Modelos de comunicación

La comunicación cambia según la parte del sistema que se esté observando. El cliente envía consultas y espera resultados, mientras que los servicios internos intercambian operaciones y mensajes de replicación. Por eso conviene separar el acceso SQL de la coordinación que ocurre dentro de la base.

3.3.1. Comunicación entre cliente y servidor

El cliente se conecta al punto de acceso SQL y trabaja mediante solicitudes y respuestas sobre la red. En la práctica se utilizó el cliente MySQL dentro de un contenedor para conectarse a TiDB Server. En YugabyteDB, YSQL ofrece una relación similar con sus clientes. La existencia de varios nodos internos no convierte toda la interacción en un modelo peer-to-peer sin roles [1], [2].

3.3.2. Comunicación interna y replicación

Cuando una operación debe modificar datos, el servidor SQL coordina el trabajo con el almacenamiento. Dentro de un grupo Raft, el líder replica cambios y necesita una mayoría para confirmar el registro correspondiente. No es necesario esperar a todos los seguidores para completar cada escritura [9], [12].

Esta coordinación combina RPC y mensajes entre servicios. Además, el consenso de una Region no sustituye el protocolo que necesita una transacción que afecta a varias Regions. Se trata de responsabilidades relacionadas, pero diferentes dentro del sistema [14].

3.3.3. Comunicación con la capa analítica

TiFlash recibe los cambios de TiKV de forma asíncrona como learner. Esto permite separar parte del trabajo analítico, aunque las lecturas deben comprobar el avance de los datos. El sistema puede necesitar esperar para ofrecer la vista correspondiente a una consulta [7].

El artículo de Ding y sus colaboradores propone Jasper sobre TiDB para estudiar el equilibrio entre aislamiento de cargas y frescura. Ayuda a comprender que mantener representaciones por filas y columnas tiene costos. Sus mejoras no se presentan aquí como resultados del TiDB 7.5.0 utilizado en la práctica [15].

3.4. Tolerancia a fallos

La tolerancia a fallos busca que el sistema pueda continuar bajo determinadas caídas. En TiDB y YugabyteDB, Raft ayuda a conservar un registro replicado, pero el funcionamiento completo también depende de otros servicios, de la red y de la distribución de las réplicas.

Para un grupo de tres votantes, la mayoría necesaria es de dos. Si cae un seguidor y el líder mantiene comunicación con otro votante, puede continuar sin una elección nueva. Si cae el líder, los supervivientes necesitan elegir otro. Esta detección y recuperación toman tiempo, por lo que no se debe prometer una respuesta inmediata ante cualquier caída [9], [12].

También existen fallos reales que afectan a varios componentes. El 13 de mayo de 2025, TiDB Cloud publicó un incidente en algunos clústeres Serverless de us-west-2 que duró 22 minutos y 34 segundos. El reporte atribuye los errores a una incompatibilidad de versiones que produjo panic en dos servidores TiKV antiguos [16].

El incidente muestra que la replicación no elimina todos los problemas operativos. El reporte no detalla los grupos Raft afectados, por lo que no corresponde calcular sus quórums a partir de esa información. Sí permite reconocer que una causa compartida puede afectar a varios servidores a la vez y superar el caso sencillo de perder un solo proceso.

4. Experimentación propia

Los conceptos anteriores se llevaron a una práctica local para observar qué pasa al detener un componente de almacenamiento. La ejecución final que respaldan las nuevas capturas se hizo con Docker Compose y TiDB 7.5.0. Los intentos previos con TiUP y Docker no se utilizan como resultados de esta corrida.

4.1. Entorno y objetivo de la prueba

El objetivo fue comprobar si se podía insertar y consultar información después de terminar uno de los tres contenedores TiKV. El archivo Compose utilizado se incluye sin modificar en `experimento/docker-compose.yml`. En él aparecen un contenedor PD, un TiDB Server y tres TiKV, todos con imágenes de versión 7.5.0.

Esta configuración simplifica el entorno para trabajar desde mi equipo con CachyOS. No incluye TiFlash, un balanceador ni varias máquinas físicas. Tampoco replica PD ni TiDB Server. Por eso, aunque permita observar una caída de TiKV, no representa un despliegue con alta disponibilidad en todas sus capas.

La hipótesis de la prueba es que, si las Regions mantienen la mayoría necesaria y los demás servicios siguen disponibles, una operación puede completarse después de perder un TiKV. Las capturas permiten revisar el estado de los contenedores y la respuesta SQL; no muestran un inventario completo de los peers de cada Region.

4.2. Desarrollo del ejercicio

El ejercicio se presenta en el orden de ejecución: levantar el entorno, preparar la base, provocar la caída y comprobar una operación posterior. Cada captura se coloca junto a la parte que explica, para que se pueda seguir la relación entre el comando utilizado y lo que realmente aparece en su salida.

4.2.1. Inicio del clúster

Primero se levantaron los servicios y se consultó su estado. En la Figura 2 se observan los contenedores `tidb-pd`, `tidb-server` y los tres `tidb-tikv`. También aparece la identificación de Carlos, el host y la fecha del equipo. El aviso sobre el campo `version` del Compose no impidió que los contenedores mostraran estado Up.

```
sudo docker compose up -d
sudo docker compose ps
```

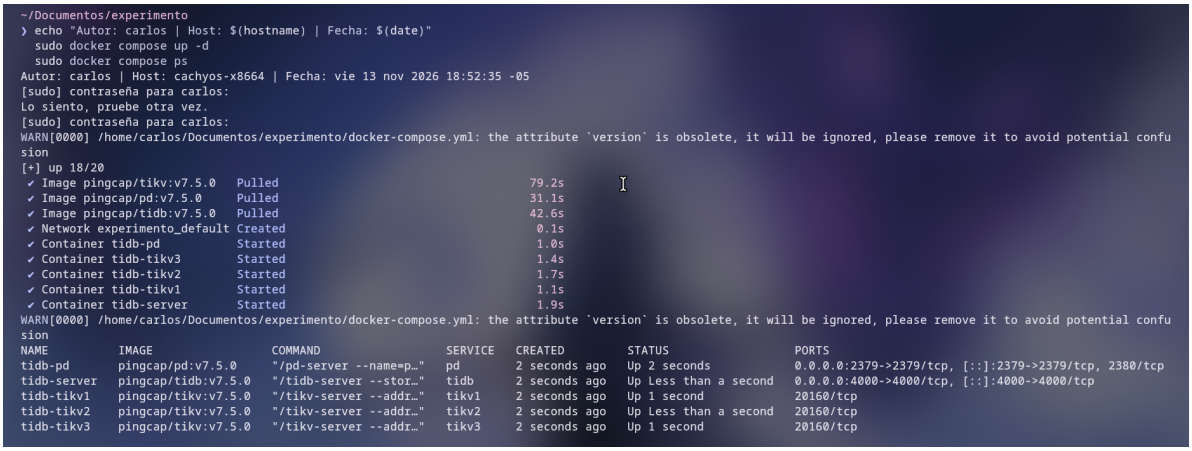


Figura 2: Inicio del entorno Docker Compose y listado de sus cinco contenedores. La captura muestra usuario, host y fecha local.

El estado Up indica que los contenedores estaban iniciados en ese momento. La conexión SQL del paso siguiente es la que permite comprobar que TiDB también estaba atendiendo consultas.

4.2.2. Preparación de la base de datos

Después se utilizó un contenedor con el cliente MySQL para conectarse al servidor TiDB mediante la red `experimento_default`. La Figura 3 muestra que el servidor respondió con la versión `8.0.11-TiDB-v7.5.0`. El nombre MySQL corresponde al cliente y al protocolo compatible; la base que atiende la conexión es TiDB.

```
sudo docker run -it --rm --network experimento_default mysql:8.0 mysql -h tidb-server -P 4000 -u
     root
```

Dentro de esa sesión se creó la base y la tabla:

```
CREATE DATABASE experimento;
USE experimento;
CREATE TABLE prueba (
  id INT AUTO_INCREMENT PRIMARY KEY,
  dato VARCHAR(100),
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

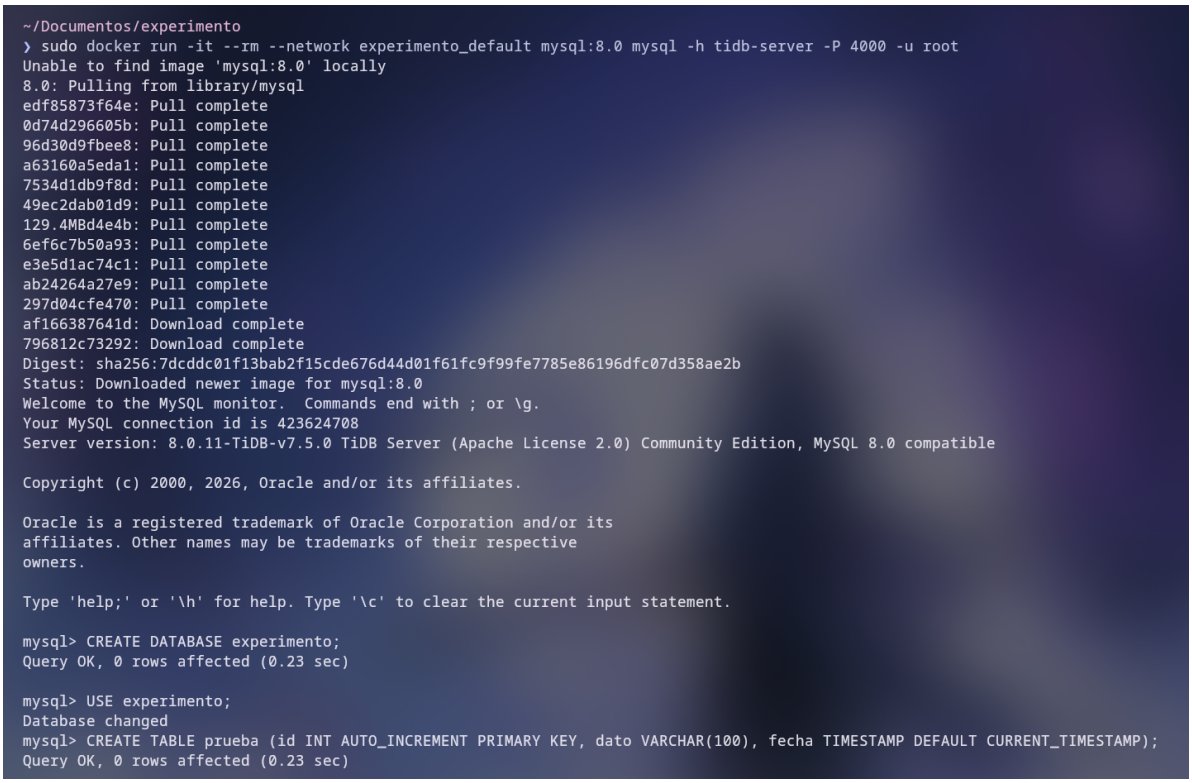


Figura 3: Conexión del cliente MySQL al servidor TiDB y creación de experimento.prueba. Las respuestas confirman la creación de la base y la tabla.

El primer dato se identifica después en la consulta final como `Dato antes de la caida`. La captura de preparación no incluye la orden que insertó esa fila, así que no se reconstruye su salida como si estuviera disponible.

4.2.3. Caída de un TiKV

Para provocar el fallo se terminó el contenedor `tidb-tikv1` y se consultó su estado. La Figura 4 muestra el comando, el nombre del contenedor y el estado `Exited (137)`. La documentación de Docker indica que `docker kill` envía SIGKILL por defecto al proceso principal del contenedor [17].

```
sudo docker kill tidb-tikv1
sudo docker ps -a --filter name=tidb-tikv1
```

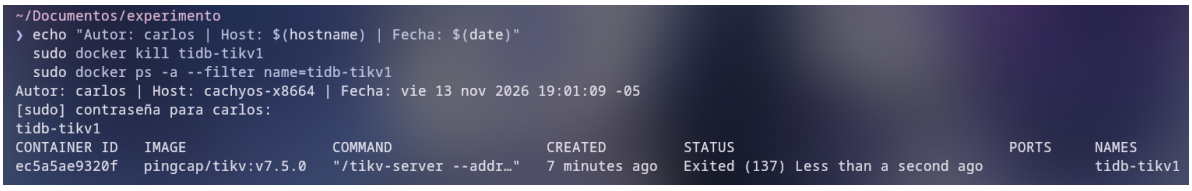


Figura 4: Terminación de tidb-tikv1 y comprobación de su estado Exited (137). La identificación registra el 13 de noviembre de 2026 a las 19:01:09, con zona local -05.

Esta acción afecta al almacenamiento, mientras PD y el servidor SQL se mantienen dentro del entorno de la prueba. La captura no indica qué Regions tenían su líder en ese contenedor; por eso no se afirma que necesariamente haya ocurrido una elección nueva.

4.2.4. Operación después de la caída

Luego se volvió a la sesión SQL y se insertó un registro diferente. La Figura 5 muestra la respuesta `Query OK, 1 row affected` y una consulta con dos filas. La primera contiene el dato anterior y la segunda el texto que identifica la inserción posterior.

```
INSERT INTO prueba (dato)
VALUES ('Insertado exitosamente tras la caida del nodo');
SELECT * FROM prueba;
```

```
mysql> -- 1. Inserta un dato demostrando que el sistema sigue operando
mysql> INSERT INTO prueba (dato) VALUES ('Insertado exitosamente tras la cada del nodo');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- 2. Verifica que los datos estn ah
mysql> SELECT * FROM prueba;
+----+-----------------------------------------------+---------------------+
| id | dato                                          | fecha               |
+----+-----------------------------------------------+---------------------+
|  1 | Dato antes de la cada                         | 2026-11-13 23:59:28 |
|  2 | Insertado exitosamente tras la cada del nodo  | 2026-11-14 00:02:53 |
+----+-----------------------------------------------+---------------------+
2 rows in set (0.00 sec)
```

Figura 5: Inserción aceptada y consulta con los registros anterior y posterior a la caída. El cliente muestra 0,01 s para el INSERT y dos filas en el SELECT.

La consulta final conserva las fechas SQL que aparecen en la imagen: `2026-11-13 23:59:28` y `2026-11-14 00:02:53`. El comando del fallo muestra otra representación horaria del equipo. Sin una consulta de las zonas horarias no se comparan directamente esos relojes ni se calcula con ellos un tiempo de recuperación.

4.3. Interpretación de lo observado

La secuencia muestra una terminación de TiKV y una inserción que fue aceptada después dentro del ejercicio presentado. Este resultado es compatible con el funcionamiento esperado cuando sobreviven las réplicas necesarias y los demás servicios continúan disponibles. Permite relacionar la replicación y la tolerancia a fallos con una operación concreta.

Sin embargo, las imágenes no muestran el estado de los peers de cada Region ni un segundo listado de contenedores junto a la consulta final. Por eso, el quórum se utiliza para explicar el mecanismo del sistema, no como una medición directa tomada durante la prueba. Tampoco se midió si existió una pausa entre el fallo y la recuperación.

Los 0,01 segundos que informa el cliente corresponden a la ejecución de ese INSERT, no al tiempo total que tardó el sistema en recuperarse. Para obtener ese último dato habría que registrar operaciones de forma continua antes, durante y después de la caída.

La prueba también tiene límites por su entorno. Todos los contenedores comparten una máquina, existe un solo PD y un solo TiDB Server, y el Compose no declara volúmenes persistentes. Se probó la caída de un contenedor, no la pérdida del equipo completo ni la conservación de datos después de eliminar los contenedores. Los comandos observados y sus resultados se documentan en `experimento/COMANDOS-Y-RESULTADOS.md`.

5. Ejemplos del entorno del estudiante

La distribución también se puede relacionar con aplicaciones que utilizo y con la experiencia de trabajar directamente sobre TiDB. Estos ejemplos permiten distinguir lo que se percibe desde una interfaz de usuario de los detalles que aparecen al revisar el funcionamiento interno de un sistema.

5.1. Banca móvil como usuario

Como usuario he utilizado banca móvil y no sé desde qué servidor se atienden mis peticiones. Para usar la aplicación no tengo que elegir un nodo ni conocer dónde se guarda la información. Esto lo relaciono con la transparencia de ubicación, porque el sistema permite acceder al servicio sin mostrar esos detalles internos.

También he observado, de forma general al utilizar aplicaciones distribuidas, que algunas veces un servicio deja de responder y la falla llega al usuario. Eso me permite entender que ocultar la ubicación y mantener el servicio disponible son cosas diferentes. En el caso de la banca móvil, mi observación se limita a su uso; no conozco su arquitectura interna ni puedo atribuirle TiDB, Raft u otra tecnología sin documentación que lo respalde.

5.2. TiDB como desarrollador durante la práctica

Al realizar el ejercicio pude ver el sistema desde otra perspectiva. Primero se preparó una tabla y después se terminó un contenedor TiKV. En la sesión SQL se aceptó una nueva inserción, como se observa en la Figura 5. Esto me ayudó a relacionar la replicación con una operación que yo podía ejecutar y consultar.

Antes de revisar los componentes, desde el cliente solo veía las consultas y sus resultados. Al trabajar con los contenedores pude identificar qué parte se estaba deteniendo y cuáles seguían formando parte del entorno. Esta experiencia me permitió comprender mejor la transparencia de fallos, junto con la necesidad de conservar suficientes réplicas y otros servicios disponibles. La prueba tuvo el alcance de una caída local de almacenamiento; no comprobó todos los fallos posibles de TiDB ni el comportamiento de YugabyteDB.

6. Análisis comparativo

TiDB y YugabyteDB resuelven el problema de distribuir una base SQL, pero organizan de forma diferente sus componentes y el manejo del tiempo. La Tabla 1 reúne ocho criterios de la unidad para revisar esas diferencias. La comparación de YugabyteDB se apoya en documentación; no se realizó una prueba equivalente de ese sistema en este trabajo.

Tabla 1: Comparación de TiDB y YugabyteDB según los conceptos de la unidad.

Criterio	**TiDB**	**YugabyteDB**
Acceso	Cliente compatible con MySQL.	YSQL vinculado con PostgreSQL.
Ubicación y movimiento	Distribuye datos en Regions; PD participa en su planificación.	Distribuye datos en tablets; YB-Master coordina catálogo y otras tareas.
Replicación	Raft por Region; TiFlash puede actuar como learner.	Raft por tablet; permite read replicas adicionales.
Concurrencia y tiempo	Versiones de datos y timestamps proporcionados por PD.	Versiones de datos y relojes lógicos híbridos.
CAP en escritura	Necesita una mayoría para confirmar en el camino consistente.	La replicación principal también necesita mayoría.
PACELC	Mantener garantías implica coordinación aun sin partición.	HLC no elimina el quórum; las lecturas asíncronas tienen otro alcance.
Comunicación	SQL cliente–servidor y comunicación interna entre servicios.	SQL cliente–servidor y comunicación entre capas y tablets.
Tolerancia a fallos	Puede continuar ante pérdidas que conserven el quórum y las demás dependencias.	Mantiene la misma necesidad de mayoría en cada tablet.

La tabla se apoya en las descripciones de almacenamiento, transacciones y replicación de ambos sistemas [2], [6], [8], [9], [12], [13]. Una coincidencia importante es que ambos utilizan Raft, así que la diferencia no se reduce a decir que uno es consistente y el otro busca responder rápido. Hay que considerar qué operación se hace, cómo están ubicadas las réplicas y qué garantías se han configurado.

TiDB presenta una separación clara entre la capa SQL, TiKV y la opción analítica TiFlash. Esto permite ampliar y estudiar cada parte con funciones distintas. YugabyteDB, por su parte, combina YSQL con DocDB y utiliza HLC para el manejo del tiempo. Esta organización puede resultar cercana a aplicaciones que ya utilizan PostgreSQL, pero la migración necesita comprobar sus funciones y comportamiento.

La práctica permite hablar de una operación local de TiDB después de una caída, pero no da datos para afirmar que supera a YugabyteDB en rendimiento. Una comparación de ese tipo necesitaría la misma carga, recursos, datos y condiciones de fallo en ambos sistemas.

7. Postura crítica

Después de revisar el diseño y la práctica, considero que TiDB ofrece herramientas útiles para distribuir una base SQL, pero su conveniencia depende del problema que se quiere resolver. Aquí se presentan las fortalezas que encuentro y los aspectos que tendría que revisar antes de elegirlo para una aplicación real.

7.1. Fortalezas del diseño

La separación de componentes ayuda a comprender dónde se procesa una consulta, dónde se guardan los datos y quién coordina su ubicación. También permite distribuir capacidad entre distintos servicios. En la práctica fue posible provocar una caída en el almacenamiento y obtener después una inserción aceptada, algo que hizo más claro el papel de las réplicas.

La opción de TiFlash resulta interesante cuando se necesita combinar transacciones y análisis. Sin embargo, mantener dos representaciones de datos tiene un costo. Song y sus colaboradores

describen problemas de sincronización y aislamiento de cargas, mientras Jasper estudia una organización adaptativa sobre TiDB para equilibrar aislamiento y frescura [3], [15]. Por eso, la ventaja analítica debe valorarse según las consultas reales de la aplicación.

7.2. Limitaciones y contextos de uso

Tener varios nodos también significa administrar más servicios y comunicaciones. Una transacción puede depender de más de un grupo, y la ubicación de las réplicas influye en el tiempo que tarda en confirmarse. Para una aplicación pequeña, que funciona bien con una base en una sola máquina, esa complejidad puede ser difícil de justificar si no existe una necesidad concreta de distribución.

Tampoco elegiría una operación de escritura basada en quórum esperando que cualquier lado de una partición pueda seguir confirmando cambios. Si el requisito es trabajar de forma independiente durante desconexiones prolongadas, se debe analizar otro diseño o ajustar las garantías esperadas. Este compromiso se relaciona directamente con lo revisado en CAP y PACELC.

El incidente de TiDB Cloud y las limitaciones del Compose local recuerdan que la disponibilidad depende de todo el entorno. En una instalación real tendría que revisar dominios de fallo, redundancia de PD y SQL, respaldos, persistencia y procedimientos de actualización. Replicar datos es una parte importante de la solución, pero no reemplaza esas decisiones operativas.

8. Conclusiones

La revisión permitió comprender que TiDB y YugabyteDB comparten mecanismos de consenso, aunque organizan de otra forma el almacenamiento y las marcas de tiempo. La diferencia entre TSO y HLC ayuda a explicar su coordinación, pero no basta para afirmar que uno siempre es más rápido o que el otro ofrece mejores garantías en cualquier situación.

En el ejercicio se levantaron un PD, un TiDB Server y tres TiKV con Docker Compose. Las capturas muestran la terminación de `tidb-tikv1` y una inserción aceptada después en la sesión SQL, donde quedaron visibles dos registros. Esto permitió relacionar la tolerancia a fallos con un resultado propio y concreto, dentro del alcance documentado.

También quedó claro que la prueba tiene límites. No se registró el estado de cada Region, no se midió de forma continua la disponibilidad y no se probaron fallos del host completo, PD o TiDB Server. Por esa razón, el resultado no se utiliza para prometer recuperación inmediata ni disponibilidad total.

Mi conclusión es que TiDB resulta una opción interesante cuando existe una necesidad real de distribuir SQL y combinar tipos de carga. Para elegirlo en un proyecto tendría que valorar esas necesidades junto con el costo de coordinación y operación. La práctica ayudó a entender ese compromiso desde el funcionamiento del sistema y no solo desde la definición de los conceptos.

9. Declaración de uso de inteligencia artificial

El informe partió de mis avances en DOCX. Utilicé OpenAI Codex, agente basado en GPT-6, para buscar y contrastar fuentes, apoyar su lectura, revisar y adaptar el texto, darle formato en LaTeX y resaltar fragmentos. Verifiqué la información con documentación oficial, registros editoriales y resultados de mi práctica. Se conservaron las capturas originales y la versión final del informe; debo poder explicar su contenido.

Referencias

[1] PingCAP. «TiDB Architecture.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tidb-architecture/

[2] Yugabyte, Inc. «Architecture.» Documentación stable, consultada como contraste arquitectónico, visitado 13 de nov. de 2026. dirección: https://docs.yugabyte.com/stable/architecture/

[3] H. Song, W. Zhou, H. Cui, X. Peng y F. Li, «A survey on hybrid transactional and analytical processing,» *The VLDB Journal*, vol. 33, n.º 5, págs. 1485-1515, 2024. DOI: 10.1007/s00778-024-00858-9 dirección: https://link.springer.com/article/10.1007/s00778-024-00858-9

[4] PingCAP. «WeBank Cuts Costs by 30 % While Scaling TiDB to Petabyte-Level Operations.» Caso de uso publicado por el proveedor a partir de una presentación del equipo de WeBank, visitado 13 de nov. de 2026. dirección: https://www.pingcap.com/case-study/webank-cuts-costs-scaling-tidb-petabytes/

[5] PingCAP. «TimeStamp Oracle (TSO) in TiDB.» Documentación oficial de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tso/

[6] PingCAP. «TiKV Overview.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tikv-overview/

[7] PingCAP. «TiFlash Overview.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tiflash-overview/

[8] Yugabyte, Inc. «Fundamentals of Distributed Transactions.» Documentación stable; sus funciones no se ensayaron localmente, visitado 13 de nov. de 2026. dirección: https://docs.yugabyte.com/stable/architecture/transactions/transactions-overview/

[9] PingCAP. «TiDB Storage.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tidb-storage/

[10] C.-Y. Huang, Q. Peng, F.-X. Zhang, S.-Y. Wang, C. Luo, Y.-F. Zhang y G. Yu, «Survey on Key Techniques of Multi-replica Distributed Transaction Processing and Representative Database Systems,» *Journal of Software*, vol. 35, n.º 1, págs. 455-480, 2024, En chino; título inglés oficial de la revista. DOI: 10.13328/j.cnki.jos.006822 dirección: https://www.jos.org.cn/html/2024/1/6822.htm

[11] PingCAP. «High Availability FAQs.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/high-availability-faq/

[12] Yugabyte, Inc. «DocDB replication layer.» Documentación stable, visitado 13 de nov. de 2026. dirección: https://docs.yugabyte.com/stable/architecture/docdb-replication/

[13] Yugabyte, Inc. «Read replicas.» Documentación stable, visitado 13 de nov. de 2026. dirección: https://docs.yugabyte.com/stable/architecture/docdb-replication/read-replicas/

[14] PingCAP. «TiDB Architecture FAQs.» Documentación de TiDB v7.5, visitado 13 de nov. de 2026. dirección: https://docs.pingcap.com/tidb/v7.5/tidb-faq/

[15] Z. Ding, X. Zhang, C. Zhang, Y. Sun, K. Xu, W. Lu y X. Du, «Breaking the Isolation-Freshness Trade-off: Joint Adaptive Storage Optimization for HTAP Systems,» *Proceedings of the VLDB Endowment*, vol. 19, n.º 6, págs. 1142-1155, 2026. DOI: 10.14778/3797919.3797924 dirección: https://www.vldb.org/pvldb/vol19/p1142-ding.pdf

[16] PingCAP. «TiDB Cloud Serverless Downtime in us-west-2 Region.» Reporte del 13 de mayo de 2025, visitado 13 de nov. de 2026. dirección: https://status.tidbcloud.com/incidents/xk1mmz7n7hkb

[17] Docker, Inc. «docker container kill.» Referencia oficial de Docker CLI, visitado 13 de nov. de 2026. dirección: https://docs.docker.com/reference/cli/docker/container/kill/

[18] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, 27 de nov. de 2024, art. 158. DOI: 10.1186/s13677-024-00716-7 visitado 13 de nov. de 2026. dirección: https://link.springer.com/article/10.1186/s13677-024-00716-7

A. Esquema inicial a mano

El bosquejo de la Figura 6 fue la base para organizar los componentes y pasar después al diagrama digital. En él se distinguen el cliente, la capa SQL, PD, TiKV y TiFlash. La fotografía original se conserva sin alterar; no muestra nombre ni fecha visibles, por lo que esa identificación de la evidencia sigue pendiente.

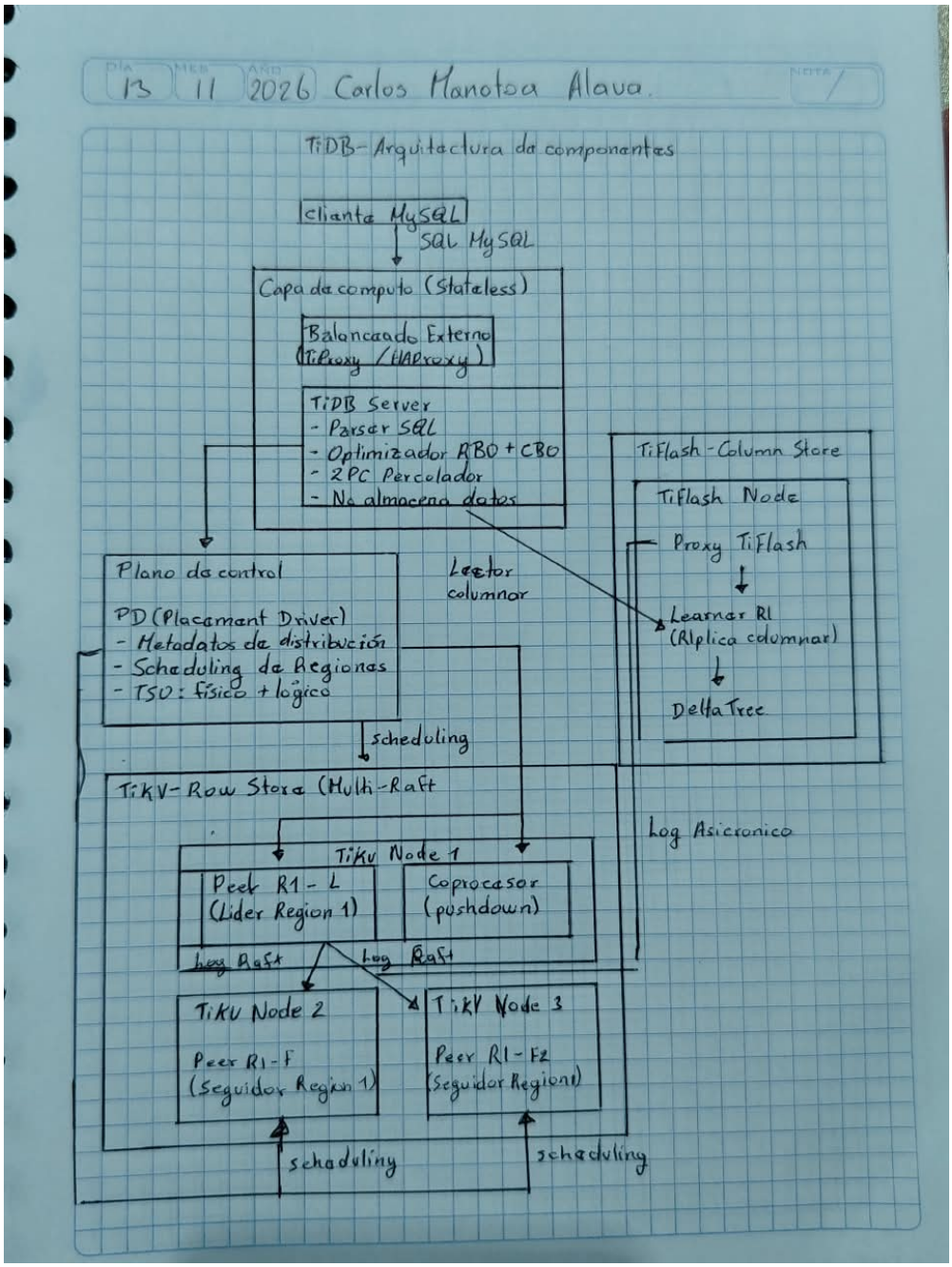


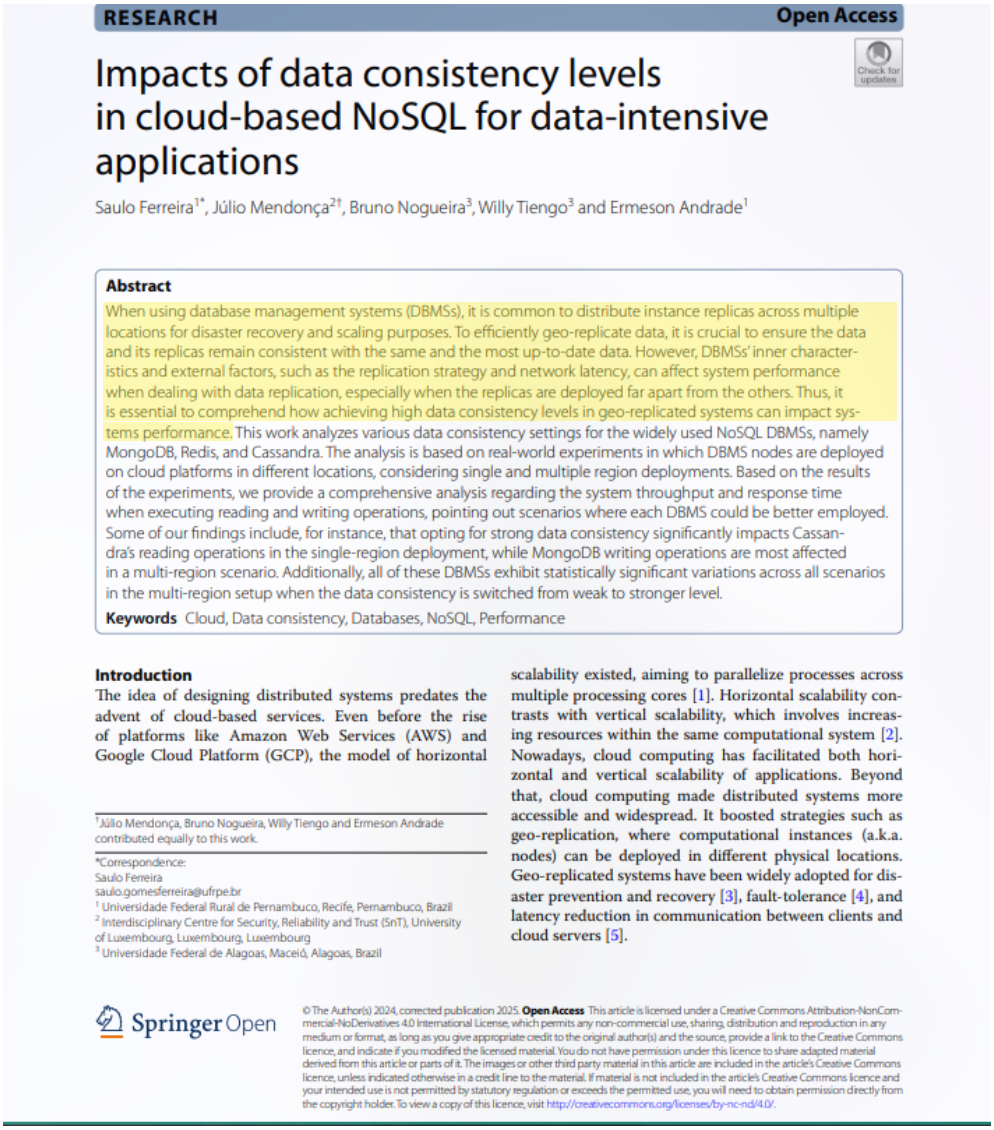
Figura 6: Boceto original de la arquitectura de TiDB, utilizado como punto de partida del diagrama del informe.

B. Evidencias de lectura de las fuentes

Las siguientes capturas relacionan tres lecturas en inglés y de acceso gratuito con las ideas del informe. El resaltado amarillo identifica los fragmentos sobre replicación, consistencia y procesamiento HTAP; cada figura indica la página correspondiente de la fuente.

B.1. Replicación, consistencia y rendimiento

El artículo de Ferreira y sus colaboradores permite comprender que mantener réplicas también requiere coordinación. En el resumen de la página 1 se explica que la estrategia de replicación, la latencia de la red y el nivel de consistencia pueden influir en el rendimiento [18]. La Figura 7 destaca esa relación. Sus experimentos estudian MongoDB, Redis y Cassandra, por lo que no se presentan como resultados de TiDB.



RESEARCH Open Access

Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications

Saulo Ferreira[1*], Júlio Mendonça[2†], Bruno Nogueira[3], Willy Tiengo[3] and Ermeson Andrade[1]

Abstract

When using database management systems (DBMSs), it is common to distribute instance replicas across multiple locations for disaster recovery and scaling purposes. To efficiently geo-replicate data, it is crucial to ensure the data and its replicas remain consistent with the same and the most up-to-date data. However, DBMSs' inner characteristics and external factors, such as the replication strategy and network latency, can affect system performance when dealing with data replication, especially when the replicas are deployed far apart from the others. Thus, it is essential to comprehend how achieving high data consistency levels in geo-replicated systems can impact systems performance. This work analyzes various data consistency settings for the widely used NoSQL DBMSs, namely MongoDB, Redis, and Cassandra. The analysis is based on real-world experiments in which DBMS nodes are deployed on cloud platforms in different locations, considering single and multiple region deployments. Based on the results of the experiments, we provide a comprehensive analysis regarding the system throughput and response time when executing reading and writing operations, pointing out scenarios where each DBMS could be better employed. Some of our findings include, for instance, that opting for strong data consistency significantly impacts Cassandra's reading operations in the single-region deployment, while MongoDB writing operations are most affected in a multi-region scenario. Additionally, all of these DBMSs exhibit statistically significant variations across all scenarios in the multi-region setup when the data consistency is switched from weak to stronger level.

Keywords Cloud, Data consistency, Databases, NoSQL, Performance

Introduction

The idea of designing distributed systems predates the advent of cloud-based services. Even before the rise of platforms like Amazon Web Services (AWS) and Google Cloud Platform (GCP), the model of horizontal scalability existed, aiming to parallelize processes across multiple processing cores [1]. Horizontal scalability contrasts with vertical scalability, which involves increasing resources within the same computational system [2]. Nowadays, cloud computing has facilitated both horizontal and vertical scalability of applications. Beyond that, cloud computing made distributed systems more accessible and widespread. It boosted strategies such as geo-replication, where computational instances (a.k.a. nodes) can be deployed in different physical locations. Geo-replicated systems have been widely adopted for disaster prevention and recovery [3], fault-tolerance [4], and latency reduction in communication between clients and cloud servers [5].

†Júlio Mendonça, Bruno Nogueira, Willy Tiengo and Ermeson Andrade contributed equally to this work.

*Correspondence:
Saulo Ferreira
saulo.gomesferreira@ufrpe.br
[1] Universidade Federal Rural de Pernambuco, Recife, Pernambuco, Brazil
[2] Interdisciplinary Centre for Security, Reliability and Trust (SnT), University of Luxembourg, Luxembourg, Luxembourg
[3] Universidade Federal de Alagoas, Maceió, Alagoas, Brazil

Springer Open

© The Author(s) 2024, corrected publication 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Figura 7: Resumen de Ferreira y colaboradores, página 1. Se resalta la relación entre replicación, latencia, consistencia y rendimiento.

B.2. Aislamiento y frescura en HTAP

El artículo de Ding y sus colaboradores presenta Jasper, una propuesta implementada sobre TiDB para estudiar el equilibrio entre aislamiento de cargas y frescura de los datos [15]. En la introducción de la página 1142 se explica que mantener datos por filas y columnas genera un costo de sincronización, mientras que utilizar una sola representación puede aumentar la competencia por recursos. La Figura 8 destaca esas ideas. Las mejoras de Jasper no se atribuyen a la instalación utilizada en mi práctica.

ABSTRACT

HTAP systems aim to support large-scale transaction processing while preserving real-time analytics over fresh operational data. Achieving this dual goal requires carefully ensuring workload isolation and data freshness. However, this is a challenge that existing systems often struggle to meet, as they rely on static, coarse-grained storage configurations, such as duplicating full data across dual storage model (row and column store) or storing data in a single model. These incur excessive synchronization overhead that degrades data freshness or compromises workload isolation under mixed workloads. To this end, we present Jasper, a joint adaptive storage mechanism that dynamically configures fine-grained storage layouts based on workload characteristics. Jasper performs workload-aware horizontal and vertical partitioning and selectively materializes column store replicas for update-sparse, query-intensive partitions. This design ensures strong workload isolation while minimizing unnecessary data redundancy, significantly reducing synchronization overhead and improving data freshness. We implement Jasper in TiDB and conduct extensive evaluations using both standard benchmarks and a real-world TiDB production workload. Extensive evaluations on both benchmarks and real-world TiDB production workloads show that Jasper cuts workload completion time by 20.43%–40.59%, delivering state-of-the-art performance in balancing isolation and freshness for HTAP systems.

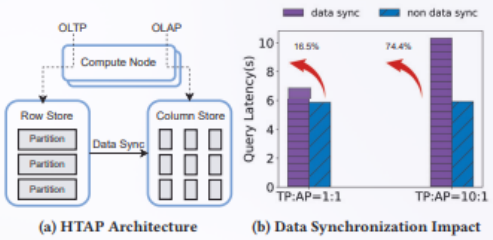
PVLDB Reference Format:
Zhenghao Ding, Xinyi Zhang, Chao Zhang, Yishen Sun, Kai Xu, Wei Lu, Xiaoyong Du. Breaking the Isolation-Freshness Trade-off: Joint Adaptive Storage Optimization for HTAP Systems . PVLDB, 19(6): 1142 - 1155, 2026.
doi:10.14778/3797919.3797924

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/DBXAI/Jasper.

*Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 19, No. 6 ISSN 2150-8097.
doi:10.14778/3797919.3797924



(a) HTAP Architecture (b) Data Synchronization Impact

Figure 1: HTAP System Characteristics.

1 INTRODUCTION

HTAP systems support large-scale transaction processing and real-time analytics. The storage model (e.g., row vs. column store) has a significant impact on the performance of different types of workloads [2, 8, 33]. Therefore, existing HTAP systems (TiDB [25], Heatwave [22], SQL Server [32], SingleStore [51]) typically support hybrid storage models and route workloads to the corresponding storage according to their characteristics to achieve high workload isolation (Figure 1a). Specifically, row stores—where data tuples are stored contiguously by row—are used for transactional processing, while column stores—where each column is stored contiguously—are employed to accelerate analytical queries.

Maintaining both row and column stores in HTAP systems introduces data synchronization overhead. To ensure consistency, systems adopt techniques such as delta merge or primary store rebuild [22, 25, 31, 32, 45] to propagate updates from the row store to the column store, which can significantly degrade performance. We conduct a micro-benchmark on TiDB using CH-Benchmark [15] to measure the impact: enabling synchronization increases query latency by 16.5% to 74.4%, with more severe degradation under heavier transactional workloads. This is because TiDB retrieves the latest data on reads [25], and recent updates not yet reflected in the column store introduce additional read latency. These findings highlight the substantial overhead of synchronization over two storage models and prompt a critical question: *is it truly necessary to maintain dual-model storage for all data?*

Storing data in a single model avoids synchronization but compromises workload isolation due to increased contention. This highlights a fundamental trade-off in HTAP systems between isolation and data freshness. Commercial systems like TiDB [25] and ByteHTAP [14] adopt coarse-grained dual-storage designs to reduce

Figura 8: Introducción de Ding y colaboradores, página 1142 (primera del PDF). Se resaltan el costo de sincronización y el compromiso entre aislamiento y frescura.

B.3. Diseños y compromisos de HTAP

La revisión de Song y sus colaboradores clasifica los diseños HTAP y explica problemas que aparecen al combinar transacciones y análisis [3]. En la página 1486 se identifican dificultades como la sincronización de datos, la consistencia y el aislamiento entre cargas. La Figura 9 destaca el párrafo que reúne esos problemas. Estas ideas permiten comprender por qué separar formatos y recursos puede ofrecer ventajas y también introducir trabajo adicional de coordinación, como se analiza en la introducción y la postura crítica.

transactions can promptly use the results of analytical queries as payloads. We summarize the common target scopes of HTAP databases in Sect. 3.

Such technical coherence between OLTP and OLAP raises new design challenges over data storage, execution engines, and query optimizers. To tackle these challenges, tremendous efforts have been made in this area, resulting in a rich literature of related papers and solutions (see Table 1). The adopted architectures and design choices may vary significantly. To systematically oversee existing designs, we broadly classify existing HTAP systems based on the design of data formats, which is also adopted by a previous study [126]. Differently, our survey greatly extends the two-way classfication of HTAP by summarizing common issues and discussing the combination of potential solutions.

In particular, we first study monolithic HTAP systems, which serve OLTP and OLAP workloads with identical data format designs. Typically, OLTP and OLAP workloads show different data access patterns. OLTP can operate on a single data tuple at a time and access many attributes, while OLAP typically accesses a massive number of rows at a time with a subset of tuple attributes. Monolithic HTAP systems strive to make a "one-size-fits-all" format design and allow OLAP to make progress regardless of concurrent OLTP. Monolithic HTAP may incur significant performance issues when serving mixed workloads. We summarize three common issues: contradictory format demand, performance degradation of MVCC, and performance isolation. In response to these common issues, several new techniques for data grouping, MVCC, in-memory snapshots, and read/write splitting have been proposed. We discuss them in Sect. 4.

We then study HTAP systems using hybrid data formats, which independently optimize the underlying data formats for OLTP and OLAP. Although hybrid data formats mitigate the contradictory format demand, they bring new performance issues. We summarize six common issues for HTAP with a hybrid data format: data synchronization, data consistency, gap of write efficiency, hybrid data access, performance isolation, and sharding strategy. To understand the design rationales, we further classify HTAP with hybrid data formats into four typical architectures: row-oriented, column-oriented, separated, and hybrid architectures. Based on our taxonomy, we discuss the existing solutions for the six common issues one by one in Sect. 5

To enlighten new HTAP designers, we make a pros and cons summary of existing solutions (monolith design, column-oriented hybrid design, row-oriented hybrid design, separated hybrid design, and hybrid architecture with hybrid data formats) in various scopes (transaction performance, query performance, storage overhead, data freshness, performance isolation, scalability, performance stability, and system complexity). This can be a good starting point for

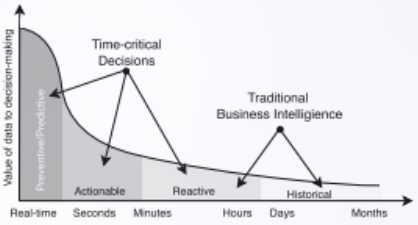


Fig. 1 A comparison between the targets of HATP databases and traditional business intelligence [35]. HTAP databases provide real-time analytics to maximize the value of data

newcomers (see Sect. 6). We also present readers with emerging new applications, benchmarks, and future directions.

1.1 Contributions

To the best of our knowledge, our survey takes the first step to deeply review the existing HTAP architectures, compare the resident technologies, and give a comprehensive overview of HTAP regarding system design, applications, and benchmarks. Our contribution includes:

- We systematically summarize the common design goals of HTAP. We then intensively study the common design issues and existing solutions to enlighten and guide researchers and practitioners in this area.
- We present a new fine-grained taxonomy, which comprehensively organizes HTAP architectures based on the storage layer. This provides us with a uniform methodology to compare different implementations and trade-offs.
- We present readers with cutting-edge real-time applications powered by HTAP systems. They can be practical templates for developing new ones and motivating new research in HTAP.
- Last but not least, we also present the emerging benchmarks, unique evaluation metrics, and standard evaluation methods of HTAP.

1.2 Related surveys and research collections

There are two tutorials related to our survey. Özcan et al. presented a tutorial on the taxonomy of HTAP systems in SIGMOD 2017 [126]. It covers multiple data processing systems, including NoSQL [157], SQL-on-Hadoop [2, 68, 162], and Spark SQL [17, 22, 109, 179]. However, due to the intensive research and development efforts, the architectures and technologies of new emerging HTAP systems (Table 1) are not covered in this tutorial (especially for the designs that appear after 2017). Compared to previous designs, these new

Figura 9: Introducción de Song y colaboradores, página 1486 (segunda del PDF). Se resaltan los problemas y compromisos de los diseños HTAP con formatos híbridos.